UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⵉⴱⵣⵔⵜ ⵏ ⵜⵣⴰⵔⵜ ⴷ ⵜⵎⴰⵡⴰⵙⵉⵏ ⵏ ⵗⴳⴰⴷⵉⵔ

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE D'AGADIR
المدرسة العليا الخاصة للهندسة بأكادير
ⵜⵉⵎⵙⵙⵓⵔⵉⵏ ⵜⵉⵏⵏⴰⵢⵏⵉⵢⵏ ⵜⵉⴱⵣⵔⵜ ⵏ ⵗⴳⴰⴷⵉⵔ

TRAVAUX PRATIQUES

Bases de Données & Modélisation des SI

Atelier 3 : Premières requêtes SQL avec SELECT et DISTINCT

Objectif : écrire et exécuter des requêtes de consultation dans la base de données AdventureWorks, choisir les colonnes utiles, documenter le code et obtenir des valeurs ou combinaisons distinctes.

Prérequis : base AdventureWorks restaurée et exploration des schémas réalisée.

Le TP comprend des rappels de cours, des manipulations guidées et un travail à rendre à la fin. Les requêtes étudiées consultent les données ; la création, la modification, les jointures et les filtres seront abordés ultérieurement.

I- Préparer l'environnement et les noms des objets

1. Connectez-vous au serveur dans pgAdmin. Développez adventureworks → Schemas → production → Tables. Repérez productcategory, productreview et product, puis leurs colonnes.

2. Repérez aussi sales.salesorderheader et person.countryregion. Notez les noms exacts : ils peuvent varier selon la conversion PostgreSQL utilisée pour la sauvegarde.

Convention des exemples : les objets sont écrits en minuscules, par exemple production.product et sellstartdate. Si votre base contient des noms délimités avec majuscules, utilisez leurs noms exacts entre guillemets doubles. Exemple : "Production"."Product" et "SellStartDate". Ne traduisez pas les identifiants en français.

II- Comprendre la structure d'une requête : SELECT

Une requête est une instruction adressée au SGBD. SELECT demande des données, la liste qui le suit précise les colonnes à restituer, FROM désigne la source et le point-virgule termine l'instruction.

```
SELECT *
FROM production.productcategory;
```

Dans cette requête, l'astérisque * demande toutes les colonnes de la table. Le point entre production et productcategory sépare le schéma du nom de la table. Si la table comporte quatre colonnes, le résultat contient ces quatre colonnes.

```
SELECT name
FROM production.product;
```

```
SELECT name, productnumber
FROM production.product;
```

La virgule sépare les colonnes de la liste SELECT. Il ne faut pas de virgule avant FROM. Sans filtre ni autre transformation, choisir deux colonnes au lieu de toutes les colonnes réduit la largeur du résultat, pas le nombre de lignes.

Une requête peut être écrite sur une ligne. Placer FROM sur une nouvelle ligne rend la lecture plus claire. Les mots-clés SQL sont écrits ici en majuscules par convention ; la couleur dépend du thème de l'éditeur et ne modifie pas l'exécution.

À faire : Sur papier, identifiez dans chaque exemple les mots-clés, les colonnes, le schéma, la table et le signe de fin d'instruction.

III- Comprendre la casse et le rôle du schéma

Les mots-clés SELECT et select ont le même sens. Dans PostgreSQL, les identifiants non délimités sont ramenés en minuscules : Product et product désignent alors product. En revanche, "Product" est un nom délimité sensible à la casse et peut désigner un autre objet.

```
select name from production.product;
SELECT name FROM production.product;
```

Ces deux requêtes sont équivalentes lorsque les objets existent sous les noms non délimités utilisés. Les guillemets doubles servent aux identifiants ; les apostrophes servent aux valeurs textuelles et ne remplacent pas ces guillemets.

Sans nom de schéma, PostgreSQL cherche l'objet dans les schémas du search_path. L'omission du schéma peut donc fonctionner ou échouer, y compris pour public. Qualifier les noms évite les ambiguïtés et facilite la lecture.

```
SELECT *
FROM production.productcategory;
```

Deux schémas peuvent contenir des tables homonymes : schema1.table_exemple et schema2.table_exemple sont alors deux objets distincts. Utilisez toujours le schéma réel dans les requêtes de ce TP.

IV- Ouvrir Query Tool et exécuter une instruction

3. Sélectionnez la base AdventureWorks, puis ouvrez Tools → Query Tool, ou utilisez le menu contextuel de la base. Vérifiez le nom de la base associé à la connexion du nouvel onglet.

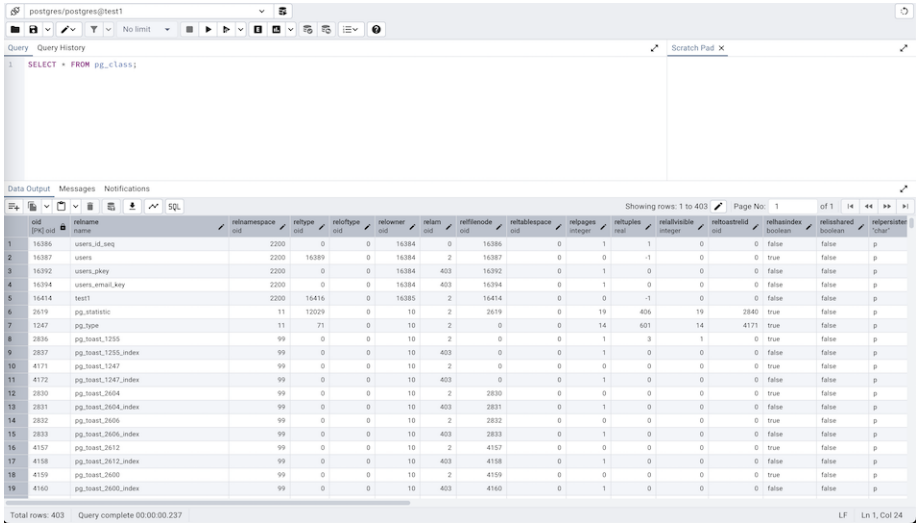


Figure 1 — Interface réelle du Query Tool dans la documentation pgAdmin

4. Repérez l'éditeur SQL, le bouton d'exécution, Data Output, Messages et Query History.

5. Saisissez la requête suivante et exécutez-la avec le bouton Execute script ou Execute query approprié. Le raccourci peut être F5 ou Alt+F5 selon l'action et la version ; consultez l'infobulle du bouton de votre installation.

```
SELECT *
FROM production.productcategory;
```

6. Observez Data Output. Repérez les noms des colonnes, leurs types lorsqu'ils sont affichés et les valeurs. Dans Messages ou la barre d'état, relevez les informations de réussite et le temps indiqué.

Résultat attendu : La catégorie de produits apparaît sous forme de tableau. Les colonnes attendues comprennent généralement productcategoryid, name, rowguid et modifieddate ; utilisez votre structure réelle.

Une grille peut charger les résultats progressivement. Une limite d'affichage, de récupération ou une instruction LIMIT peut restreindre ce que vous voyez : ne supposez pas qu'une limite universelle de 1 000 lignes s'applique à toutes les versions.

V- Observer une erreur de résolution de table

7. Saisissez puis exécutez la requête suivante, sans le schéma. Cette manipulation sert à comprendre le chemin de recherche.

```
SELECT *
FROM productcategory;
```

8. Si une erreur « relation does not exist » apparaît, ouvrez Messages et relevez le nom de la relation et la position ou la ligne signalée.

9. Réexécutez la version qualifiée production.productcategory. Si elle échoue, vérifiez la base connectée, l'orthographe, les guillemets et les noms exacts dans l'explorateur.

Une erreur de relation n'implique pas que les données sont perdues : le nom, le contexte de connexion, les droits ou le chemin de recherche peuvent être en cause. Corrigez un élément à la fois et relisez le message.

VI- Consulter une autre table et choisir les colonnes

10. Remplacez uniquement le nom de la table pour consulter les avis sur les produits.

```
SELECT *
FROM production.productreview;
```

À faire : Comparez avec productcategory : nombre et noms de colonnes, présence d'un commentaire long, nombre de lignes réellement retournées ou récupérées.

11. Consultez les colonnes de production.product, puis exécutez la sélection du seul nom des produits.

```
SELECT name
FROM production.product;
```

Résultat attendu : Une seule colonne apparaît. Plusieurs produits peuvent être retournés ; sélectionner une colonne ne signifie pas sélectionner une seule ligne.

12. Ajoutez le numéro de produit, puis inversez l'ordre des colonnes et comparez les en-têtes.

```
SELECT name, productnumber
FROM production.product;
```

```
SELECT productnumber, name
FROM production.product;
```

L'ordre des expressions après SELECT détermine l'ordre des colonnes affichées. Cette opération ne change pas les tables. Elle ne trie pas les lignes : aucun ordre de lignes n'est garanti sans ORDER BY, notion étudiée plus tard.

13. Répétez volontairement la colonne name et observez le résultat.

```
SELECT productnumber, name, name
FROM production.product;
```

La même valeur peut apparaître dans deux colonnes du résultat. Les deux en-têtes name sont alors identiques. Cette répétition est autorisée mais apporte peu ici ; elle servira ultérieurement à comparer des expressions ou traitements.

Choisir uniquement les colonnes utiles réduit le volume de données transmis et facilite la lecture. Cela peut améliorer les performances, mais le temps dépend aussi du volume, du cache, des index et du contexte : une exécution isolée ne suffit pas à démontrer un gain.

VII- Ajouter et lire les commentaires SQL

Un commentaire documente le code et n'ajoute aucune colonne au résultat. Deux tirets -- commentent la suite de la ligne. Un bloc commence par /* et se termine par */ ; il peut contenir plusieurs lignes.

14. Exécutez cette requête après avoir vérifié le nom de sellstartdate.

```
-- Consulter le début de commercialisation des produits
SELECT productnumber, name, sellstartdate
FROM production.product;
```

15. Remplacez le commentaire par un bloc et ajoutez un commentaire en fin de ligne.

```
/* Consultation des produits
   et de leur date de début de commercialisation. */
SELECT productnumber, name, sellstartdate -- date de début de vente
FROM production.product;
```

Résultat attendu : Les commentaires restent visibles dans l'éditeur et ne figurent pas comme texte dans Data Output. Une valeur NULL éventuelle signifie une information absente ; elle n'est pas égale à une date vide ni à zéro.

Attention : tout ce qui suit -- sur la même ligne est ignoré. Si FROM est placé après -- sur cette ligne, il est lui aussi commenté et la requête peut devenir incorrecte. Fermez toujours le bloc avec */.

VIII- Enregistrer le script et retrouver une requête

16. Enregistrez le contenu de l'éditeur dans un fichier TP3_Nom_Prenom.sql avec le bouton Save ou Save As. Séparez les instructions par des points-virgules et ajoutez un commentaire avant chaque exemple.

17. Ouvrez Query History. Sélectionnez une requête réussie et utilisez Copy to Query Editor, ou l'action équivalente de votre version. Vérifiez le texte recopié avant de l'exécuter.

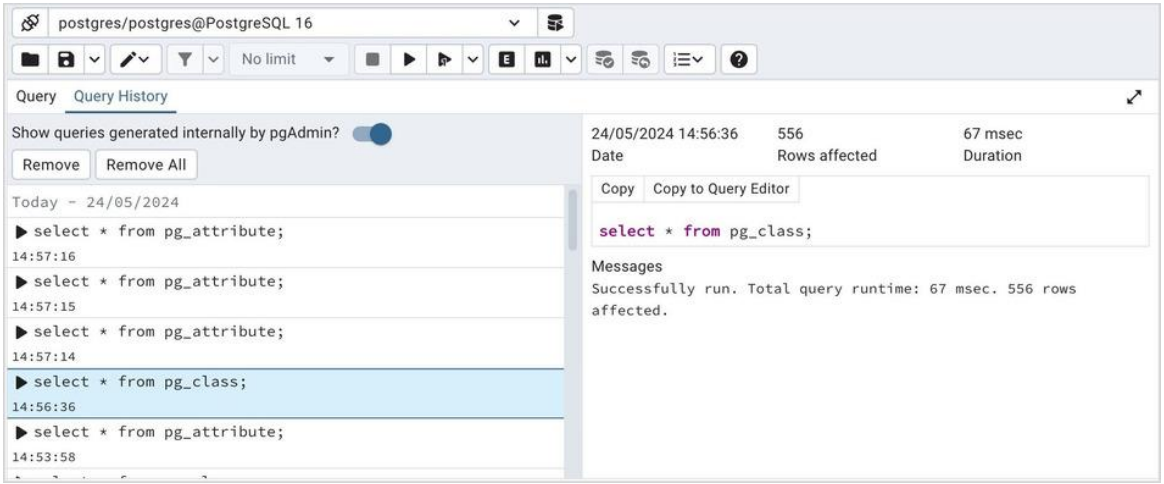


Figure 2 — Historique réel des requêtes dans pgAdmin

L'historique peut être vide au premier démarrage et sa conservation dépend de la configuration. Le fichier .sql reste votre copie de travail. Quand plusieurs instructions sont présentes, sélectionnez exactement celle que vous voulez tester avant de l'exécuter avec l'action adaptée.

Résultat attendu : Vous retrouvez une requête antérieure, l'exécutez à nouveau et conservez un script lisible contenant les instructions de la séance.

IX- Comprendre SELECT DISTINCT

SELECT conserve normalement toutes les lignes du résultat, y compris les répétitions. DISTINCT élimine les doublons sur l'ensemble des colonnes sélectionnées. Il agit sur le résultat de lecture et ne supprime aucune ligne des tables.

```
SELECT DISTINCT color
FROM production.product;
```

Avec une seule colonne, chaque valeur différente apparaît une seule fois. Pour DISTINCT, plusieurs valeurs NULL dans cette colonne ne produisent qu'une ligne NULL. Les couleurs et leur nombre dépendent des données de votre sauvegarde.

Exemple pédagogique indépendant d'AdventureWorks : si une colonne contient Rouge, Rouge, Bleu, Bleu et Vert, la sélection classique contient cinq lignes ; la sélection DISTINCT en contient trois. Leur ordre n'est pas garanti. DISTINCT ne signifie pas « conserver la première occurrence dans l'ordre de saisie ».

18. Comparez les deux requêtes suivantes et relevez le nombre de colonnes et de lignes dans chaque résultat, si toutes les lignes ont été récupérées.

```
SELECT color
FROM production.product;
```

```
SELECT DISTINCT color
FROM production.product;
```

Expliquez ce qui change et ce qui ne change pas entre les deux résultats.

X- Rechercher les clients présents dans les commandes

19. Dans sales → Tables, repérez salesorderheader et vérifiez les colonnes customerid et salespersonid. Exécutez d'abord la sélection classique des identifiants de clients.

```
SELECT customerid
FROM sales.salesorderheader;
```

20. Ajoutez DISTINCT et comparez les résultats.

```
SELECT DISTINCT customerid
FROM sales.salesorderheader;
```

La première requête fournit un identifiant de client par ligne de commande au sens d'en-tête de commande. La seconde fournit chaque identifiant de client présent dans ces commandes une seule fois. Un client ayant passé plusieurs commandes peut donc figurer plusieurs fois dans le premier résultat.

Dans le message, environ 31 465 en-têtes de commande et 19 119 clients distincts sont cités. Ce sont des repères propres à un jeu de données, pas des valeurs à recopier ou à imposer à votre base.

XI- Appliquer DISTINCT à plusieurs colonnes

Avec plusieurs colonnes, DISTINCT élimine les combinaisons identiques. Il ne rend pas chaque colonne unique séparément. Il compare les valeurs du couple, sans effectuer une concaténation textuelle.

Exemple : les couples (Client 1, Vendeur 10), (Client 1, Vendeur 10) et (Client 1, Vendeur 20) deviennent deux couples distincts. Le Client 1 apparaît encore deux fois car il est associé à deux vendeurs différents.

21. Exécutez les deux sélections et comparez les couples client–vendeur.

```
SELECT customerid, salespersonid
FROM sales.salesorderheader;
```

```
SELECT DISTINCT customerid, salespersonid
FROM sales.salesorderheader;
```

customerid et salespersonid sont des identifiants, pas les noms des personnes. Retrouver les noms nécessitera des relations et jointures étudiées ultérieurement. Une valeur NULL dans salespersonid indique qu'aucun identifiant de vendeur n'est renseigné sur cette ligne ; ce n'est pas le vendeur numéro zéro.

Deux lignes (même customerid, NULL) sont considérées comme doublons par DISTINCT. Deux lignes ayant le même client et des vendeurs différents restent distinctes. Le nombre de couples peut donc être supérieur au nombre de clients distincts.

22. Si vous devez contrôler le nombre exact de couples, adaptez le contrôle fourni.

XII- Vérifier les noms de pays ou régions

23. Dans le schéma person, repérez countryregion et ses colonnes. Exécutez les deux requêtes suivantes.

```
SELECT name
FROM person.countryregion;
```

```
SELECT DISTINCT name
FROM person.countryregion;
```

Si les deux résultats complets contiennent le même nombre de lignes, aucun doublon sur la colonne name n'a été éliminé. Cela ne démontre pas que toutes les colonnes de la table sont uniques, ni que chaque pays est effectivement représenté parmi les clients : countryregion est un référentiel de pays et régions.

XIII- Diagnostiquer les erreurs courantes

Relation inexistante : vérifiez la base, le schéma, le nom exact et les guillemets. Colonne inexistante : consultez Columns et corrigez l'identifiant. Erreur près de FROM : recherchez une virgule superflue ou une colonne mal séparée. Bloc de commentaire non terminé : ajoutez */. Résultat inattendu : vérifiez la sélection de texte exécutée, le DISTINCT et les colonnes demandées.

Dans PostgreSQL, SELECT name productnumber FROM production.product peut être interprété comme une colonne name renommée productnumber, plutôt que comme deux colonnes. Pour demander deux colonnes, écrivez bien SELECT name, productnumber. Vérifiez toujours les en-têtes du résultat.

Atelier 3 : Travail pratique en classe

1. Via la table department du schema humanresources, trouvez le nombre de départements présents dans l'entreprise.
2. Via cette même table determinez le nombre de groupe de départements différents sont présents.
3. Affichez un tableau à une colonne avec les noms de categories de produits présents dans la table productcategory du schema production.
4. Affichez un tableau avec les différentes couleurs proposés par la sociétés pour ses produits, via la table product du schema production.
5. Affichez les combinaisons uniques de productsubcategoryID avec les couleurs de produits, via la table product du schema production

Atelier 3 : Travail pratique à rendre

Mission : produire un script de consultation du catalogue et des commandes AdventureWorks. Chaque instruction doit être numérotée, commentée et terminée par un point-virgule. Les noms doivent correspondre à votre base de données.

Exercice 1 — Lire et corriger des requêtes

1. Expliquez le rôle de SELECT, FROM, *, de la virgule et du point-virgule.
2. Corrigez la requête A pour afficher deux colonnes.
3. Corrigez la requête B pour utiliser le schéma de production.

4. Expliquez quand les guillemets doubles sont nécessaires dans PostgreSQL.

```sql
-- A : afficher nom et numéro du produit
SELECT name productnumber
FROM production.product;

-- B : afficher les catégories du schéma production
SELECT *
FROM productcategory;
```

Pour B, la requête sans schéma peut réussir selon search_path : justifiez malgré toute la version qualifiée. Pour A, expliquez pourquoi l'absence de virgule peut produire une seule colonne au lieu d'un message d'erreur.

Exercice 2 — Sélectionner et présenter les produits

Écrivez et exécutez les cinq requêtes suivantes :

Q1. Toutes les colonnes des avis sur les produits.

Q2. Le nom de chaque produit uniquement.

Q3. Le numéro, puis le nom de chaque produit.

Q4. Le nom, le numéro et la date de début de commercialisation.

Q5. Le numéro suivi de deux occurrences de la colonne nom.

Ajoutez un commentaire d'une ligne à Q3 et un commentaire de bloc à Q4. Pour Q2 et Q3, expliquez si le nombre de colonnes et le nombre de lignes changent. Joignez une capture du code et du résultat de Q4.

Exercice 3 — Obtenir des valeurs et couples distincts

Écrivez et exécutez les requêtes suivantes :

Q6. Toutes les couleurs des produits, avec les répétitions.

Q7. Les couleurs distinctes des produits.

Q8. Les identifiants distincts des clients ayant une commande.

Q9. Les couples distincts identifiant client–identifiant vendeur présents dans les commandes.

Q10. Les noms distincts du référentiel des pays et régions.

Présentez un tableau comparant les nombres de lignes avec et sans DISTINCT pour les couleurs, les clients et les couples client–vendeur. Distinguez un total vérifié d'un simple nombre de lignes chargées dans la grille. Vous pouvez adapter les requêtes COUNT(*) fournies dans la partie guidée.

Expliquez pourquoi un client peut apparaître plusieurs fois dans Q9. Expliquez aussi le traitement des couples répétés contenant NULL et pourquoi l'ordre des lignes n'est pas garanti. Joignez une capture du résultat de Q9 avec son code.

Exercice 4 — Utiliser l'historique et interpréter

1. Récupérez une requête réussie depuis Query History et copiez-la dans l'éditeur. Ajoutez un commentaire indiquant son objectif, puis exécutez-la. Joignez une capture de l'historique ou documentez son indisponibilité et utilisez votre script sauvegardé.

2. Expliquez la différence entre supprimer des doublons dans un résultat et supprimer des lignes d'une table.

3. Expliquez pourquoi une sélection de deux colonnes ne limite pas, à elle seule, le nombre de lignes.